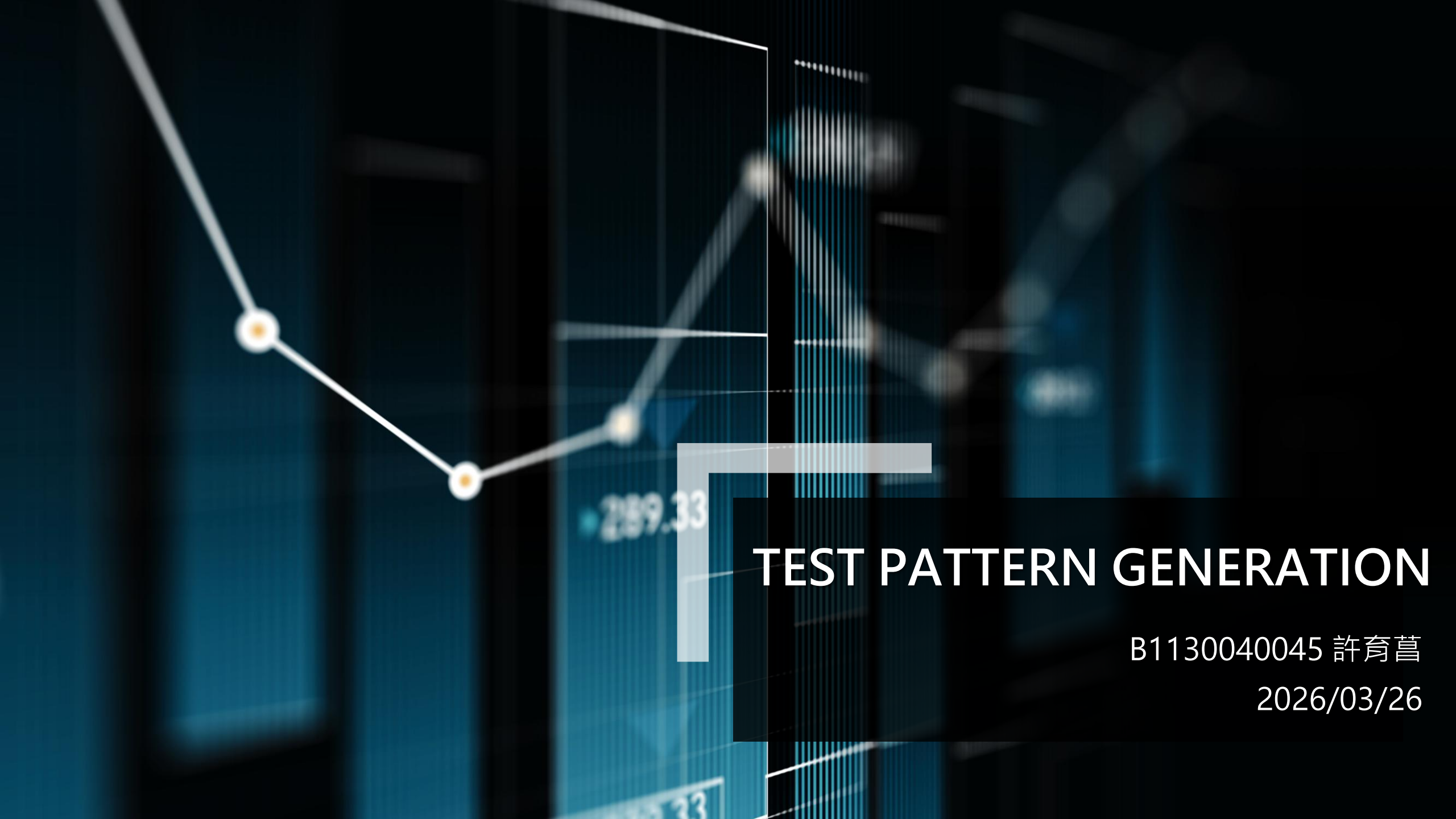




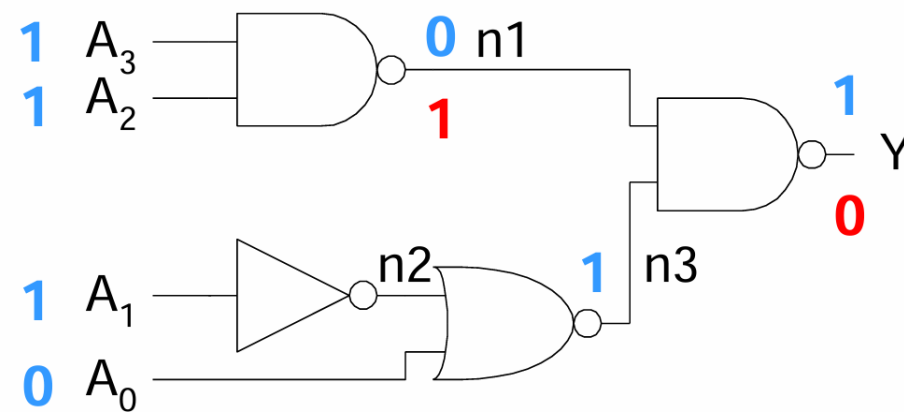
TEST PATTERN GENERATION

B1130040045 許育菀

2026/03/26

題目說明

- 給定一個電路，某些輸入可以用來檢測電路中的某些邏輯閘是否發生故障，例如：右圖的輸入為1100，正常的輸出為1；而因為n1閘發生故障(stuck at 1)，導致輸出變成0，因此1100為可以判斷n1閘是否stuck at 1的輸入。
- 在此lab中，我們需要撰寫一個程式，來**找到能檢測右圖電路是否發生故障 (stuck at 1 or stuck at 0) 的Minimum Test Pattern Set**。



程式邏輯

1、找出0(0000) ~ 15(1111)的輸入可以偵測哪一個邏輯閘是否發生stuck at 1 或 stuck at 0

使用右移符號來賦值給每個bit

```
for(int i = 0; i < 16; i++)  
{  
    a3 = (i >> 3) & 1;  
    a2 = (i >> 2) & 1;  
    a1 = (i >> 1) & 1;  
    a0 = i & 1;  
}
```

程式邏輯

2、將上一步得到的結果儲存進二維陣列中，可以獲得每一種錯誤能由哪一些輸入檢測出

【Node state to input number matrix】			0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
SA 1 A3	SA1		0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0
SA 2 A3	SA0		0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0
SA 3 A2	SA1		0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0
SA 4 A2	SA0		0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0
SA 5 A1	SA1		1	0	0	0	1	0	0	0	1	0	0	0	0	0	0	0
SA 6 A1	SA0		0	0	1	0	0	0	1	0	0	0	1	0	0	0	0	0
SA 7 A0	SA1		0	0	1	0	0	0	1	0	0	0	1	0	0	0	0	0
SA 8 A0	SA0		0	0	0	1	0	0	0	1	0	0	0	1	0	0	0	0
SA 9 n1	SA1		0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0
SA10 n1	SA0		0	0	1	0	0	0	1	0	0	0	1	0	0	0	0	0
SA11 n2	SA1		0	0	1	0	0	0	1	0	0	0	1	0	0	0	0	0
SA12 n2	SA0		1	0	0	0	1	0	0	0	1	0	0	0	0	0	0	0
SA13 n3	SA1		1	1	0	1	1	1	0	1	1	1	0	1	0	0	0	0
SA14 n3	SA0		0	0	1	0	0	0	1	0	0	0	1	0	0	0	0	0
SA15 Y	SA1		0	0	1	0	0	0	1	0	0	0	1	0	0	0	0	0
SA16 Y	SA0		1	1	0	1	1	1	0	1	1	1	0	1	1	1	1	1

邏輯閘錯誤 對 輸入數字 表

程式邏輯

3、接著儲存每個輸入可以偵測出來的邏輯閘錯誤，在程式碼中以index來表示一種邏輯閘錯誤，e.g. index 0 代表 A3 stuck at 1、index 9 代表 n1 stuck at 0

【Node state to input number matrix】																	
		0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
SA 1 A3	SA1	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0
SA 2 A3	SA0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0
SA 3 A2	SA1	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0
SA 4 A2	SA0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0
SA 5 A1	SA1	1	0	0	0	1	0	0	0	1	0	0	0	0	0	0	0
SA 6 A1	SA0	0	0	1	0	0	0	1	0	0	0	1	0	0	0	0	0
SA 7 A0	SA1	0	0	1	0	0	0	1	0	0	0	1	0	0	0	0	0
SA 8 A0	SA0	0	0	0	1	0	0	0	1	0	0	0	1	0	0	0	0
SA 9 n1	SA1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0
SA10 n1	SA0	0	0	1	0	0	0	1	0	0	0	1	0	0	0	0	0
SA11 n2	SA1	0	0	1	0	0	0	1	0	0	0	1	0	0	0	0	0
SA12 n2	SA0	1	0	0	0	1	0	0	0	1	0	0	0	0	0	0	0
SA13 n3	SA1	1	1	0	1	1	1	0	1	1	1	0	1	0	0	0	0
SA14 n3	SA0	0	0	1	0	0	0	1	0	0	0	1	0	0	0	0	0
SA15 Y	SA1	0	0	1	0	0	0	1	0	0	0	1	0	0	0	0	0
SA16 Y	SA0	1	1	0	1	1	1	0	1	1	1	0	1	1	1	1	1

```
for(int i = 0; i < 16; i++){
    cout << "SA" << setw(2) << i+1 << " " << table[i];
    for(int j = 0; j < 16; j++){
        cout << " " << matrix[i][j];

        if(matrix[i][j]) canDetect[j].push_back(i);
    }
    cout << '\n';
}
cout << '\n';
```

程式邏輯

4、以greedy演算法，找出近似的**Minimum Test Pattern Set**，在此題中剛好為最佳解

步驟為：

1. 取目前可以偵測出最多種錯誤的輸入
2. 遍歷其餘輸入可以偵測出來的錯誤，如果這個錯誤已經可以被當前所取的輸入偵測出來，就把它刪除掉。刪除後代表這個輸入可以偵測出來的錯誤數量減少
3. 重複1. 2.步驟，直到所有錯誤都可以被偵測出來，即找到近似的**Minimum Test Pattern Set**，時間複雜度為 $O(N \cdot F^2)$ ，其中 N 為輸入的數量， F 為需偵測的邏輯閘錯誤數量

演算法比較

演算法	Exhaustive attack	Dynamic programming	Greedy
優點	寫法簡單易懂，可求得最佳解	可求得最價解的解法中，速度最快的	速度最快的演算法，能處理所有電路，目前市場多數採用此演算法
缺點	無法處理有太多輸入的電路	無法處理大型電路，尤其是當電路有過多邏輯閘時	只能求得近似解
時間複雜度	$O(F \cdot 2^N)$	$O(N \cdot 2^F)$	$O(N \cdot F^2)$



實驗結果

選取輸入6

Choose 6, newStateThisRound: 0 5 6 9 10 13 14

- Input number 10: 2 5 6 9 10 13 14
- Input number 2: 5 6 9 10 13 14
- Input number 0: 4 11 12 15
- Input number 4: 4 11 12 15
- Input number 8: 4 11 12 15
- Input number 14: 1 3 8 15
- Input number 3: 7 12 15
- Input number 7: 7 12 15
- Input number 11: 7 12 15
- Input number 1: 12 15
- Input number 5: 12 15
- Input number 9: 12 15
- Input number 12: 15
- Input number 13: 15
- Input number 15: 15

After delete and sort (no output zero size element)

- Input number 0: 4 11 12 15
- Input number 4: 4 11 12 15
- Input number 8: 4 11 12 15
- Input number 14: 1 3 8 15
- Input number 3: 7 12 15
- Input number 7: 7 12 15
- Input number 11: 7 12 15
- Input number 1: 12 15
- Input number 5: 12 15
- Input number 9: 12 15
- Input number 10: 2
- Input number 12: 15
- Input number 13: 15
- Input number 15: 15

選取輸入0

```
Choose 0, newStateThisRound: 4 11 12 15
```

```
- Input number 4: 4 11 12 15  
- Input number 8: 4 11 12 15  
- Input number 14: 1 3 8 15  
- Input number 3: 7 12 15  
- Input number 7: 7 12 15  
- Input number 11: 7 12 15  
- Input number 1: 12 15  
- Input number 5: 12 15  
- Input number 9: 12 15  
- Input number 10: 2  
- Input number 12: 15  
- Input number 13: 15  
- Input number 15: 15  
- Input number 2:
```

```
After delete and sort ( no output zero size element )
```

```
- Input number 14: 1 3 8  
- Input number 3: 7  
- Input number 7: 7  
- Input number 11: 7  
- Input number 10: 2
```

選取輸入14

Choose 14, newStateThisRound: 1 3 8

- Input number 3: 7
- Input number 7: 7
- Input number 11: 7
- Input number 10: 2
- Input number 4:
- Input number 8:
- Input number 1:
- Input number 5:
- Input number 9:
- Input number 12:
- Input number 13:
- Input number 15:
- Input number 2:

After delete and sort (no output zero size element)

- Input number 3: 7
- Input number 7: 7
- Input number 11: 7
- Input number 10: 2

選取輸入3和10

```
Choose 3, newStateThisRound: 7
```

```
- Input number 7: 7  
- Input number 11: 7  
- Input number 10: 2  
- Input number 4:  
- Input number 8:  
- Input number 1:  
- Input number 5:  
- Input number 9:  
- Input number 12:  
- Input number 13:  
- Input number 15:  
- Input number 2:
```

```
After delete and sort ( no output zero size element )
```

```
- Input number 10: 2
```

```
Choose 10, newStateThisRound: 2  
Find Minimum Test Pattern Set!
```

最終結果

```
【 Minimum Test Pattern Set】  
6 0 14 3 10
```

實驗心得

我覺得這次實驗最麻煩的部分在於怎麼儲存「哪些input number可以偵測出哪些邏輯閘錯誤」，選定好了儲存方法，後面的演算法實作就很簡單了。我在一開始的時候選擇用map三維陣列進行儲存，結果我花了10分鐘就列出Step 2要求的表格，卻花了一個多小時才把Minimum Test Pattern Set找出來。

而回到家裡後，我改成用現在程式碼的code進行撰寫，30分鐘就解決了，且也把效能提升了許多。

目前的程式有個地方可以再改進，就是可以讓使用者自訂電路，不過這還界是一筆大工程了，未來有時間可以嘗試看看。